



Intelligent Control Process

A disciplined operating model for review-driven, safe, and traceable delivery

Structured execution across changes, reviews, and verification — blending governance, AI-assisted workflows, and human oversight. Designed for current and future project structures, including a .control directory model.

Intake · Planning & Review · Implementation · Verification · Deployment · Closeout

Why an Intelligent Control Process Matters

Control that is decision-aware — not control for control's sake

What it does

- Captures work before it becomes drift
- Converts review feedback into structured, tracked actions
- Aligns scope, ownership, risk, and validation
- Improves continuity across sessions and project phases
- Reduces rework and avoids untracked decisions

The goal is not control for control's sake

It is intelligent control:

- decision-aware
- traceable
- reviewable

Review feedback becomes structured, tracked, validated work — not scattered notes.

Session 0 — AI Usage Standards and Risk Controls

Set before the control process begins — these apply to every session and every phase

1

Code protection

Client code stays in approved environments — never pasted into unsanctioned tools or unapproved services.

2

Secrets redaction

Credentials, keys, connection strings, and client identifiers are redacted before any prompt or upload.

3

Approved providers only

Private / enterprise model endpoints only — no consumer browser AI (e.g., Gemini from a browser).

4

Model selection

The model follows the project standard — not personal preference or whatever is at hand.

Standards and regulations: client security standards and regulatory requirements take precedence over all defaults.

Process Architecture

The control process spans the full lifecycle



Human judgment is embedded at the decision points — not treated as a separate agenda item.

Repository and Workflow Structure

Solo project: .control at the repo root · larger projects: a paired control repository

Starting point — .control in the repo root

```
.control/  
  README.md    SESSION_START.md  
  CHANGE_TRACKER.md  DECISION_LOG.md  
  NEXT_SESSION_ISSUES.md  
  CHANGE_TYPES/  REVIEW_LOOPS/  
src/  tests/  infra/  docs/
```

Fine for a solo-dev project. But every working branch can also change the .control state — feature, recovery, and experimental branches legitimately diverge, while the team still needs one authoritative control record.

Authoritative — paired control repository

```
project/  
  src/  tests/  infra/  docs/  
project.control/  
  .control/  
    README.md    SESSION_START.md  
    CHANGE_TRACKER.md  DECISION_LOG.md  
    NEXT_SESSION_ISSUES.md  
    CHANGE_TYPES/  REVIEW_LOOPS/
```

e.g. pizzint-demo/ + pizzint-demo.control/ — governance stays close to the work without being subordinate to every application branch; authority comes from its own history, permissions, review rules, and release rhythm.

The repository is not the process — it is the durable interface through which humans, automation, and AI assistants participate in the same operating model.

Intake and Issue Tracker Creation

The intake gate for intelligent control

Session start and branching

- Start with the session objective and the change list
- Create a working branch from main before implementation
- Descriptive branch naming: feature/, fix/, docs/, chore/
- Include the primary CHG ID when available

Issue tracker rules

- Check for an existing issue in the relevant tracker first
- If none exists, create one before coding
- Link the issue record in the tracker
- Record ownership and issue source

Required notes

```
issue: #123  
issue_source: existing | created  
owner: @github-handle
```

Issue tracker systems

- GitHub · TeamCity · custom issue/ticket systems

2 Change Tracking and Status Model

The tracker is the operational source of truth

Each item records

- ID · Change · Type · Priority · Status
- Dependencies · Issue · Issue Source
- Owner · Notes

Operational model

- Move the highest-value items to active early
- Keep a visible backlog for the next session
- Update status as work progresses

Status values

proposed

active

review

blocked

completed

dropped

This keeps the process continuous and visible.

3 Planning, Review, and Decision Governance

Review is built into the lifecycle — this is how the process becomes intelligent rather than reactive

Before coding

Before merge

After validation

The process includes

- Structured prompts and review loops
- Scope clarity
- Acceptance criteria
- Rollback guidance
- Decision documentation

Decision log

When choices affect product direction, architecture, or workflow:

- Record the decision in `DECISION_LOG.md`
- Capture why the decision was made
- Preserve the reasoning for future sessions and teams

Code Creation and Test Creation

Changes are not only written — they are proven

Code creation discipline

- Scope remains aligned to the issue
- Backward compatibility preserved unless explicitly approved
- Low-risk, high-impact work is prioritized
- Provider/domain naming remains consistent
- Rollback guidance included for meaningful changes

Test creation requirements

- Tests live in the top-level tests folder
- Unit, integration, and regression coverage
- Smoke checks for user-visible behavior
- Validate both success and recovery paths

Verification and Deployment Gate

Where operational reality meets controlled execution

Before merge or release, confirm

- Validation is complete
- Tracker is current
- Issue references are present
- Decision log is updated if needed
- Rollback guidance exists
- Blockers are cleared

Deployment is a governance decision

— not just a technical action. Confirm:

- Risk
- Readiness
- Rollback path
- Human ownership of the rollout

6 Manual Verification and Closeout

A continuous delivery rhythm rather than isolated changes

Manual verification closes the loop

- Smoke tests
- End-to-end workflow check
- User-path validation
- Regression spot checks
- Confirmation of real-world outcome

Closeout — at session end

- Summarize completed, active, blocked, and proposed items
- Carry forward unresolved work
- Update the next session queue
- Ensure the backlog remains prioritized

Core Principles of the Intelligent Control Process

Informed · adaptive · disciplined · evidence-based

1 **Traceability** before implementation

2 **Ownership** before execution

3 **Review** before merge

4 **Validation** before closeout

5 **Rollback readiness** before risk

6 **Human accountability** at each decision point

7 **Continuous queue management** across sessions

This is intelligent control: informed · adaptive · disciplined · evidence-based

Intelligent Control Process

A structured system for turning review feedback into safe, traceable, validated work

It combines

Repository governance

Issue tracker discipline

Branch and ownership control

Design review

Implementation tracking

Test creation

Verification

Deployment decision

Human accountability

Session 0 standards always apply: approved environments · secrets redacted · private providers · project-standard models · client regulations first

The human is key to the intelligent control process — we own every decision point, and our clients see us actively engaged: clarity, governance, evidence, and safe execution.